

MySQL

基本过程：输入sql语句，完成表格操作

先用workbench熟悉一下

sql语法

库：

```
-- 1. 查看所有数据库
SHOW DATABASES;

-- 2. 创建数据库
CREATE DATABASE 库名; (若已存在会报错)
-- 安全写法：不存在才创建
CREATE DATABASE IF NOT EXISTS student_db;

-- 3. 选择/进入数据库（必须执行，不然建表会报错No database selected）->也可以图形化界面双击
USE student_db;

-- 4. 删除数据库
DROP DATABASE IF EXISTS student_db;
```

表：

```
-- 1. 查看当前库里面所有表
SHOW TABLES;

-- 2. 创建表（咱们学生表例子）
CREATE TABLE student(
    id INT PRIMARY KEY AUTO_INCREMENT COMMENT '学生编号，主键自增',
    --COMMENT备注说明
    --PRIMARY KEY唯一标识，id不能重复
    --AUTO_INCREMENT自增，只能搭配数字主键使用，自动id+1

    name VARCHAR(20) NOT NULL COMMENT '姓名',
    --VARCHAR是可变字符串，(20)最多20个字符

    age INT COMMENT '年龄',

    gender CHAR(1) COMMENT '性别'
    --CHAR(1)固定长度1
);

-- 3. 查看表结构（表头数据类型）
DESC student;

-- 4. 删除整张表
DROP TABLE IF EXISTS student;
```

查

```
-- 查询表中所有列所有数据，“*”代指全部列
SELECT * FROM student;

-- 只查指定列（姓名、年龄）
SELECT name, age FROM student;

-- 条件查询：只查年龄=18的学生
SELECT * FROM student WHERE age = 18;
--“=”精确匹配

-- 模糊查询：名字带"张"的学生
SELECT * FROM student WHERE name LIKE '%张%';
--LIKE是模糊匹配
--通配符（模糊）->“%”代指多个字符（可以是0个），“_”代指一个
```

插

```
-- 插入一条完整数据
INSERT INTO student(name,age,gender,address,phone)
VALUES ('王五',20,'男','渝中区','13512345678');
VALUES()--可以接多行
```

改

```
-- 修改张三的年龄为19
UPDATE student SET age=19 WHERE name='张三';
```

删

```
-- 删除名字叫李四的这一行记录
DELETE FROM student WHERE name='李四';
```

补充

1. WHERE：条件筛选，用来过滤数据，= 大于 > 小于 <
2. PRIMARY KEY：主键，唯一标识一行，不能重复、不能为空
3. AUTO_INCREMENT：自增，id 自动 + 1，不用手动填写
4. NOT NULL：不能为空
5. COMMENT：注释，只是说明文字，不参与运行
6. --：单行注释，后面写说明，MySQL 不会执行这一行文字
7. UPDATE、DELETE 一定要加 WHERE 条件，否则清空整张表！
8. 语句末尾英文分号；

数据类型

int, double之外，还支持date (XXXX-XX-XX) , DECIMAL(m,n) (共m位, n位小数)

建库

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

books_db

Tables

book

Columns

book_id

book_name

author

publisher

publish_date

price

stock

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

world

Administration Schemas

Information

Table: book

Columns:

book_id int AI PK

book_name varchar(100)

author varchar(50)

publisher varchar(50)

publish_date date

price decimal(5,2)

stock int

Query 1

Limit to 1000 rows

```

1 create database books_db;
2 use books_db;
3
4 CREATE TABLE book(
5     book_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '书籍编号; 主键自增',
6     book_name VARCHAR(100) NOT NULL COMMENT '书名',
7     author VARCHAR(50) NOT NULL COMMENT '作者',
8     publisher VARCHAR(50) NOT NULL COMMENT '出版社',
9     publish_date DATE NOT NULL COMMENT '出版日期',
10    price DECIMAL(5,2) NOT NULL COMMENT '价格',
11    stock INT NOT NULL COMMENT '库存'
12);
13
14 INSERT INTO book(book_name,author,publisher,publish_date,price,stock)
15 VALUES
16 ('西游记','吴承恩','人民文学出版社','2019-05-01',39.80,20),
17 ('朝花夕拾','鲁迅','人民教育出版社','2018-08-18',25.50,15),
18 ('海底两万里','凡尔纳','译林出版社','2020-03-15',32.00,12);
19
20 SELECT * FROM book;
21
22
23
24

```

Result Grid

book_id	book_name	author	publisher	publish_date	price	stock
1	西游记	吴承恩	人民文学出版社	2019-05-01	39.80	20
2	朝花夕拾	鲁迅	人民教育出版社	2018-08-18	25.50	15
3	海底两万里	凡尔纳	译林出版社	2020-03-15	32.00	12

book 2 x

Apply Revert Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	11:27:29	CREATE TABLE book(book_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '书籍编号; 主键自增', book_name...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for th...	0.000 sec
21	11:27:33	CREATE TABLE book(book_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '书籍编号; 主键自增', book_name...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for th...	0.000 sec
22	11:27:57	CREATE TABLE book(book_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '书籍编号; 主键自增', book_name...	0 row(s) affected	0.032 sec
23	11:28:17	INSERT INTO book(book_name,author,publisher,publish_date,price,stock) VALUES ('西游记','吴承恩','人民文学出版社','2019...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
24	11:28:17	SELECT * FROM book LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

查询:

```

2
3 • SELECT * FROM book WHERE book_name LIKE '西%';
4

```

Result Grid

Filter Rows:

Edit: Export/Import: Wrap Cell Content:

book_id	book_name	author	publisher	publish_date	price	stock
1	西游记	吴承恩	人民文学出版社	2019-05-01	39.80	20
NULL	NULL	NULL	NULL	NULL	NULL	NULL

修改:

update book set price=49.80 where book_id=1;

book_id	book_name	author	publisher	publish_date	price	stock
1	西游记	吴承恩	人民文学出版社	2019-05-01	49.80	20

删除:

delete from book where book_id=1;

	book_id	book_name	author	publisher	publish_date	price	st
▶	2	朝花夕拾	鲁迅	人民教育出版社	2018-08-10	25.50	15
	3	海底两万里	凡尔纳	译林出版社	2020-03-15	32.00	12
★	NULL	NULL	NULL	NULL	NULL	NULL	NUL

思考

以id为唯一标识可以避免书名相同混淆的情况

查找修改还是比较方便的

就是开了安全模式必须要以唯一标识为地址去修改删除

Docker

和maven有点像，可以拉取别人的东西

```
docker pull 仓库地址/命名空间（作者名）/镜像名/版本
```

-d 后台运行

-p 端口映射

-v 绑定内外目录

-rm 删除容器

-rmi 删除镜像

-f 强制

```
2558 x 1529 APTOP-006GV0H0 x + -
Microsoft Windows [版本 10.0.26200.9457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\GUN_zyk>wsl
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker pull mysql:8.0
8.0: Pulling from library/mysql
Digest: sha256:7dcddc91f13bab2f15cde676d44d91f61fc9f99fe7785e86196dfc07d358ae2b
Status: Image is up to date for mysql:8.0
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker run -d --name mysql-user-demo -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=user_db -v /mysql_data:/var/lib/mysql mysql:8.0
b7ee08fab8ff2ff5dd2ac98a5fbf7d312c919f170ec640d2427e4b0a7f9a885c
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker exec -it mysql-user-demo bash
bash-5.1# mysql -uroot -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE user_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> CREATE TABLE users(
  ->   id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'id',
  ->   username VARCHAR(50) NOT NULL COMMENT '',
  ->   age INT COMMENT ''
  -> );
ERROR 1050 (42S01): Table 'users' already exists
mysql> INSERT INTO users(username,age) VALUES ('zhangsan',20);
Query OK, 1 row affected (0.06 sec)

mysql> SELECT * FROM users;
+----+-----+-----+
| id | username | age |
+----+-----+-----+
| 1 | zhangsan | 20 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> exit;
Bye
bash-5.1# exit
exit
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker rm -f mysql-user-demo
mysql-user-demo
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker run -d --name mysql-user-demo -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=user_db -v /mysql_data:/var/lib/mysql mysql:8.0
1c7812f9c485d3c5d973d78bd49141fe4c11887a64c148db5f3a7293a1af3a21
gun_zyk@APTOP-006GV0H0:/mnt/c/Users/GUN_zyk$ docker exec -it mysql-user-demo bash
bash-5.1# mysql -uroot -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE user_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM users;
+----+-----+-----+
| id | username | age |
+----+-----+-----+
| 1 | zhangsan | 20 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> |
```

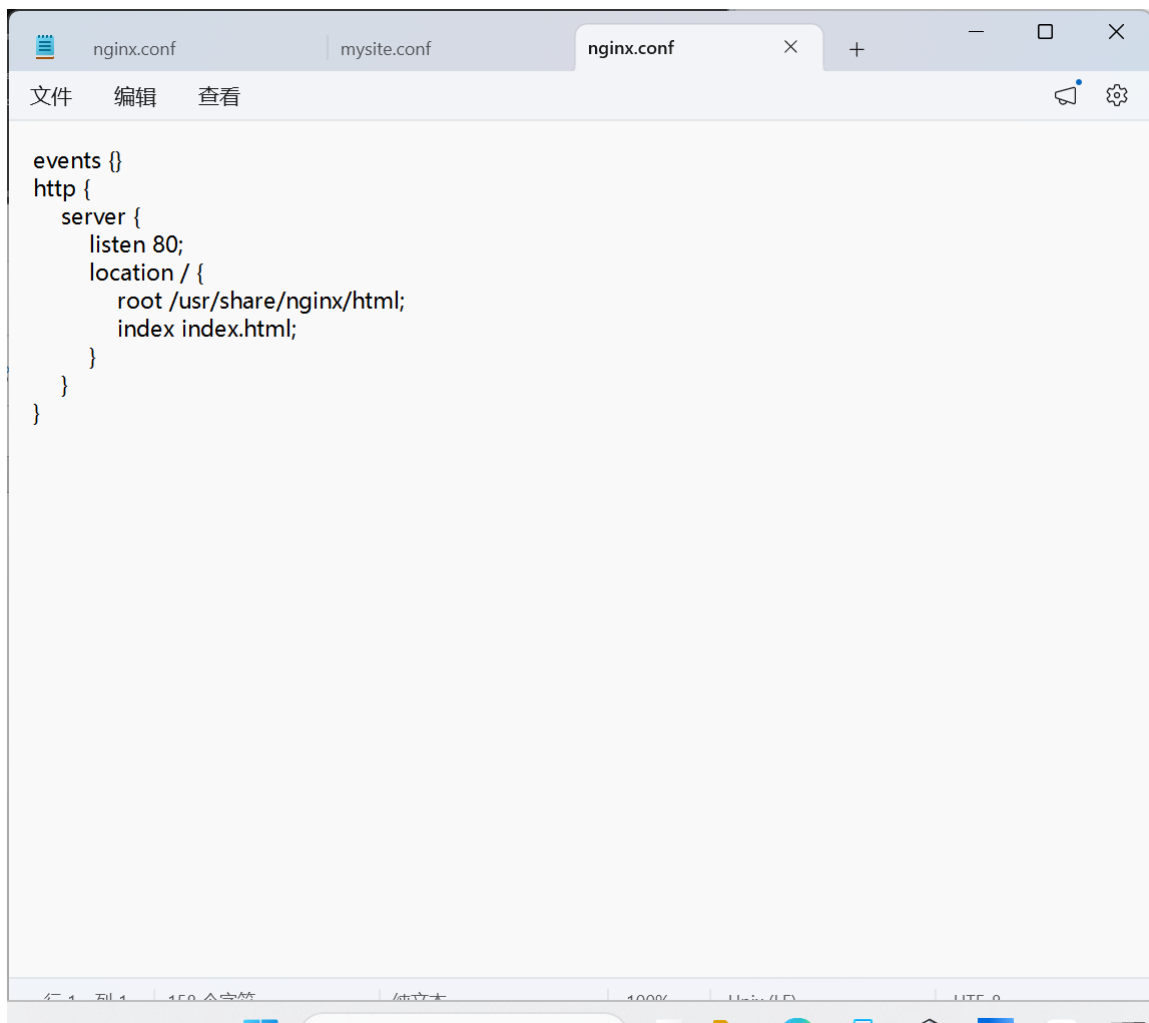
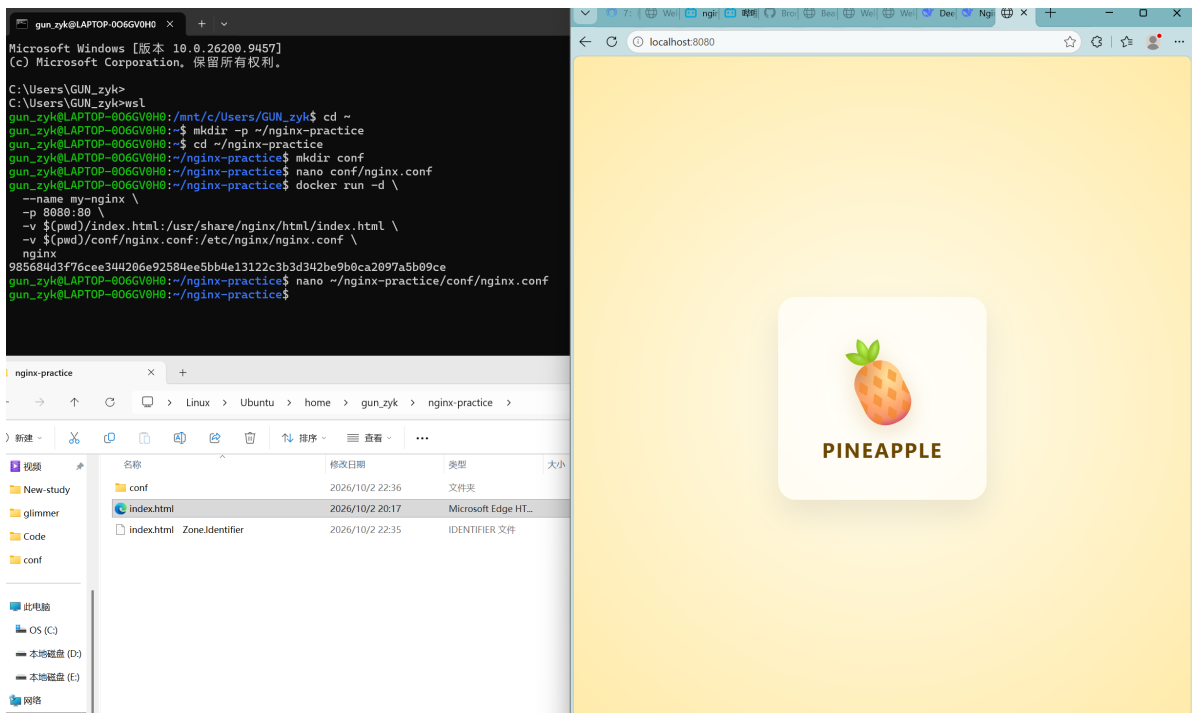
出现的问题

只有在对应层才能完成对应操作，window-->linux-->容器-->sql

数据卷不会和容器一起消失，但是存储位置在linux的虚拟空间内，不能从windows读取

现在还是不明白java项目，数据库是怎么连起来的，感觉容器之间有点割裂，而且现在只会建sql容器

学了这两个东西的一些用法，但是不会实际应用



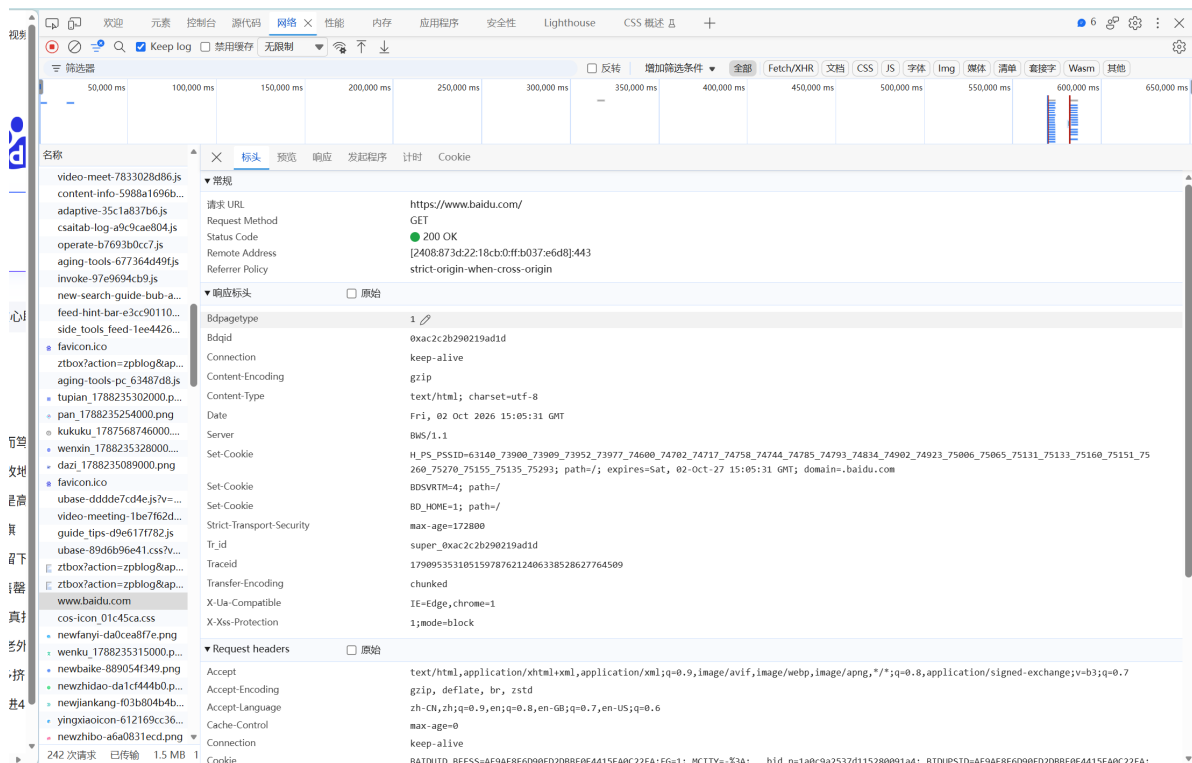
现在还是有点懵，这个练习对Nginx的理解还很浅显

我的理解是把容器里的配置文件映射到wsl里改了，本来默认有一份html的

HTTP

T1

一种通信格式，不论系统，只要格式正确即可通信



- Method: GET
- 请求 Header 信息：
 - User-Agent：浏览器标识
 - Accept：客户端能接收的数据类型
 - Host：目标服务器域名
 - Cookie：网站存储在本地的用户信息
 - Accept-Encoding：支持的压缩格式
- 服务器返回内容：HTML 网页源码、状态码 200 OK、响应头、网页资源。
- 一次 HTTP 请求组成
 - 请求行：Method + URL + HTTP 版本，表达请求方法、资源地址
 - 请求头 (Headers)：键值对，附加请求元信息
 - 空行：分隔请求头与请求体
 - 请求体 (Body)：POST 等方法携带的数据，GET 一般没有请求体
- 常见 HTTP 请求方法
 - GET：查询 / 获取资源，无请求体，参数放 URL
 - POST：提交数据、新建资源，携带请求体
 - PUT：更新资源
 - DELETE：删除资源
 - HEAD：只获取响应头，不要响应体，用于探测

T2

GEThttps://www.baidu.com/

Send

DocsParamsAuthorizationHeaders 8 •BodyScriptsSettings

Cookies

Headers7 hidden

Key	Value	Description	Bulk Edit	Presets	
✓ Student-Name	2026090914012				
Key	Value	Description			

BodyCookies 4Headers 13Test Results

200 OK • 148 ms • 1.04 KB •

HTMLPreviewVisualize

```
1 <html>
2
3 <head>
4   <script>
5     location.replace(location.href.replace("https://", "http://"));
6   </script>
7 </head>
8
9 <body>
10   <noscript>
11     <meta http-equiv="refresh" content="0;url=http://www.baidu.com/">
12   </noscript>
13 </body>
14
15 </html>
```

Header可以识别后续body的编码类型

HTTP 请求没有 Header，基础简单请求可以处理，但很多场景不行

常见的header还有content_type，标注请求体数据类型

T3

POST

https://httpbin.org/post

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Text

1

hello http

Body

Cookies

Headers 7

Test Results

200 OK

1.78 s

730 B

{ } JSON

Preview

Visualize

1

{

2

"args": {},

3

"data": "hello http",

4

"files": {},

5

"form": {},

6

"headers": {

7

"Accept": "*/*",

8

"Accept-Encoding": "gzip, deflate, br",

9

"Content-Length": "10",

10

"Content-Type": "text/plain",

11

"Host": "httpbin.org",

12

"Postman-Token": "59895268-c00b-464c-a0c7-1ac4951a7a63",

13

"User-Agent": "PostmanRuntime/2.10.1",

14

"X-Amzn-Trace-Id": "Root=1-6ac091a6-3c7470771b6dd98764594283"

15

},

16

"json": null,

17

"origin": "27.15.126.219",

18

"url": "https://httpbin.org/post"

19

}

Scanning..

POST

https://httpbin.org/post

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

Headers 8 hidden

Key	Value	Description	Bulk Edit	Presets	
Content-Type	application/json				
Key	Value	Description			

Body

Cookies

Headers 7

Test Results

200 OK • 1.53 s • 820 B • ...

{ } JSON

Preview

Visualize

1 {

2 "args": {},

3 "data": {"username": "Tom", "age": 18},

4 "files": {},

5 "form": {},

6 "headers": {

7 "Accept": "*",

8 "Accept-Encoding": "gzip, deflate, br",

9 "Content-Length": "43",

10 "Content-Type": "application/json",

11 "Host": "httpbin.org",

12 "Postman-Token": "e55e326b-f5d7-41dd-8c4f-b5ae171068ea",

13 "User-Agent": "PostmanRuntime/2.10.1",

14 "X-Amzn-Trace-Id": "Root=1-6ac091fe-7b7b2cd45b2edd4855c4e216"

15 },

16 "json": {

17 "age": 18,

18 "username": "Tom"

19 },

20 "origin": "27.15.126.219",

提交数据需要向服务器提供数据，所以是post

设置了content_type服务器才知道请求体里面的编码类型，然后解码

T4

POST ▼ Send ▼

Docs Params Authorization Headers 10 ● Body ● Scripts Settings Cookies

Headers ◀ 8 hidden

Key	Value	Description	Bulk Edit	Presets	...
☒ Content-Type	application/json				
☒ X-Student-Name	2026090914012				
Key	Value	Description			

Body Cookies Headers 7 Test Results ↺ 200 OK 1.26 s 888 B 🔍 ...

{} JSON ▶ Preview 🖼️ Visualize ▼ 🔍 📄 🔗

```
1 {
2   "args": {},
3   "data": "{\r\n  \"username\": \"admin\\\", \r\n  \"password\": \"123456\\\" \r\n}",
4   "files": {},
5   "form": {},
6   "headers": {
7     "Accept": "*/*",
8     "Accept-Encoding": "gzip, deflate, br",
9     "Content-Length": "56",
10    "Content-Type": "application/json",
11    "Host": "httpbin.org",
12    "Postman-Token": "800bae6f-03cb-433b-bbc3-f6ee2dab2012",
13    "User-Agent": "PostmanRuntime/2.10.1",
14    "X-Amzn-Trace-Id": "Root=1-6ac0924e-4724132347abbe6804c16605",
15    "X-Student-Name": "2026090914012"
16  },
17   "json": {
18     "password": "123456",
19     "username": "admin"
20   },
21 }
```

登录要向服务器提交账号密码所以post

Content-Type: application/json标注请求体的编码类型

URL是跟在IP后暴露的，隐私信息应该存在请求体里

思考

获取验证码要先向服务器说明向哪部手机发验证码，所以要post手机号

多次get服务器不会有变化，但是多次post要生成不一样的验证码

对每一个post附上一个token验证post是否完成，若未完成且重复提交直接屏蔽多余提交

浏览器获取 CSS 和 JavaScript 文件也是通过 HTTP GET 请求